

Functional Specification Document (FSD)

Private Couple Interactive Game Platform ("CoupleGame")

Document Type: Enterprise Functional Specification Document **Audience:** Product, BA, UI/UX, Frontend, Backend, QA, DevOps, Database, Security, Admin Portal (future)

1. Document Control

Field	Value
Document Title	Couple Game Platform — Functional Specification Document
Version	1.0 (MVP Baseline)
Status	Draft for Engineering Review
Owner	Product / BA
Distribution	Product, Engineering, Design, QA, DevOps, Security
Change Control	Any change to Sections 16–43 requires Architecture + Security sign-off

2. Purpose

This document is the single source of truth for the design, build, and operation of a mobile-first, private, two-player couple interaction game. It defines product scope, user journeys, business rules, data model, APIs, real-time events, security/privacy controls, and non-functional requirements sufficient for independent teams (UI/UX, frontend, backend, DB, QA, DevOps) to build without further clarification.

3. Product Vision

A private, consent-first digital space where two adult partners can strengthen connection through light trivia and paired, mutually-agreed activities — delivered with the polish of a premium product and the privacy discipline of a security-sensitive application.

4. Objectives

- O-1: Let two partners start a private, secure session in under 60 seconds.
- O-2: Guarantee that no third party can join, view, or infer session content.
- O-3: Keep every paired activity fully optional, mutual, and free of penalty for declining.
- O-4: Ship an MVP that is architecturally extensible (modes, question sources, challenge sources, accounts, admin) without a rewrite.
- O-5: Maintain a premium, non-explicit, non-pornographic visual and content standard throughout.

5. Scope (MVP)

Private room creation/join via secure link, lightweight identity (no registration), 18+ confirmation, mutual ready-state, two game modes (Love Making, Roleplay), mutual intensity selection, 10 knowledge questions per player, wrong-answer → 4-option Challenge Engine, independent scoring, end-of-game report, session/game history, real-time sync, responsive mobile-first web UI.

6. Out of Scope (MVP)

Registered accounts/login, admin portal UI, AI-generated questions/challenges, media (photo/video/audio) capture or storage, payments, push notifications, native mobile apps, multi-couple/group play, public matchmaking, social sharing of

report content, more than two game modes.

7. Stakeholders

Product Owner, Engineering Lead, UI/UX Lead, Backend Lead, Frontend Lead, DBA, Security/Privacy Officer, QA Lead, DevOps/SRE, (future) Trust & Safety, (future) Content/Admin Ops.

8. Personas

- **Room Creator ("Player A")** — initiates a session, shares the link.
- **Joining Partner ("Player B")** — joins via link, mirrors Player A's flow.
- **Returning/Reconnecting Player** — either player recovering from a dropped connection.
- **Future Admin** — curates questions/challenges, monitors non-sensitive analytics.

9. Assumptions

- Both users are on the same logical "couple" — the system does not verify real-world relationship status beyond mutual link possession and self-attested consent.
- Users primarily play on mobile browsers, often on the same Wi-Fi/same room, sometimes remote.
- No app-store distribution in MVP; installable as a responsive web app only.
- English-only content in MVP; content model supports future localization.

10. Dependencies

Secure token generation library, WebSocket infrastructure, PostgreSQL, Redis, TLS termination, hosting/CDN, (future) AI provider for question/challenge generation.

11. Terminology

Term	Definition
Room	A private two-seat container identified by a secure token
Session/Game	One complete play-through inside a Room
Round	One question cycle (both players answer their respective Question N)
Challenge	A paired, optional, mutually-bounded activity triggered by an incorrect answer
Intensity	Mutually agreed comfort ceiling: Soft / Playful / Intimate
Effective Intensity	min(Player A intensity, Player B intensity)

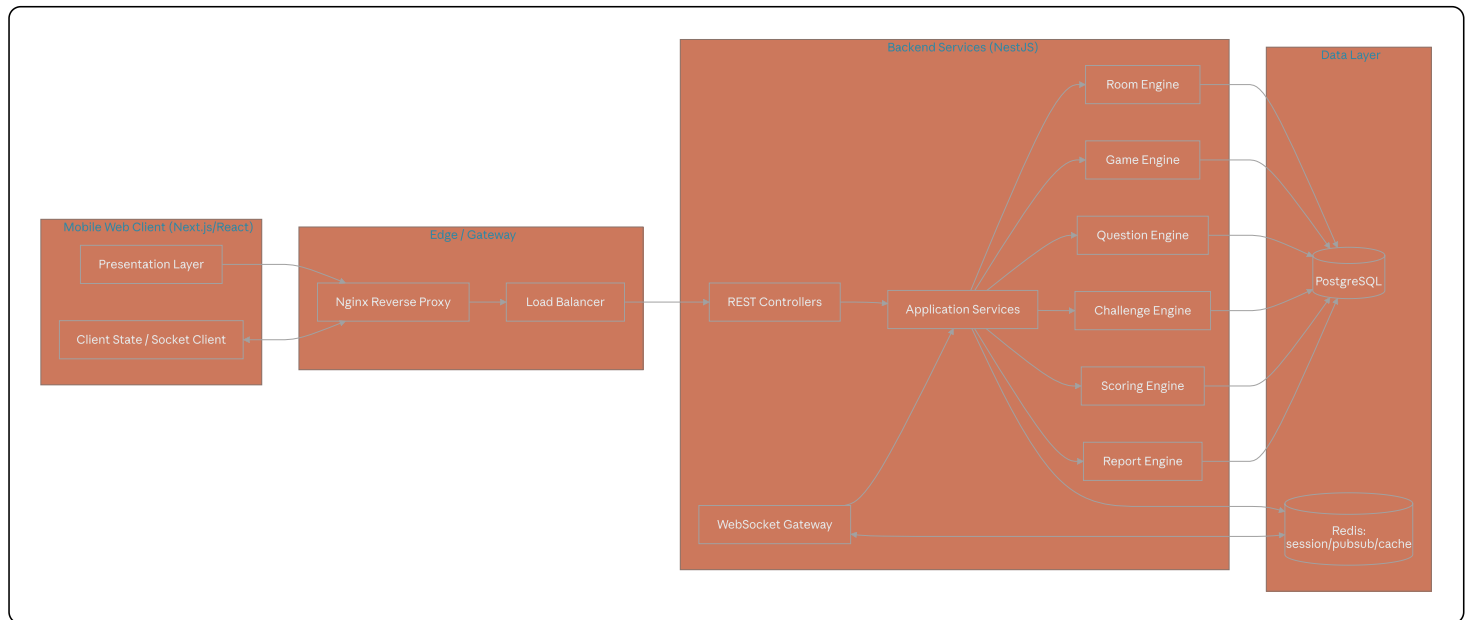
12. High-Level User Journey

Player A: Landing → Nickname → 18+ Confirm → Create Room → (optional PIN) → Get Link → Share → Waiting Room → Partner Connected → Ready → Mode Select → Intensity Select → Game → Report.

Player B: Open Link → Nickname → 18+ Confirm → (PIN if set) → Join → Waiting Room → Partner Connected → Ready → Mode Select (respond) → Intensity Select → Game → Report.

Both paths converge at "Both Ready" → synchronized game start.

13. System Architecture



Rationale:

- *Next.js/React + TypeScript* — SSR for fast first paint on mobile 4G, strong typing shared with backend.
- *NestJS* — modular DI architecture maps 1:1 to the Engine boundaries below (Room/Game/Question/Challenge/Scoring/Report), easing future extensibility.
- *PostgreSQL* — relational integrity for scores, history, and referential game state; mature JSONB support for flexible report payloads.
- *Redis* — ephemeral room/session state, WebSocket pub/sub across instances, rate limiting.
- *Socket.IO* — reconnection handling and room-based broadcast primitives out of the box.
- *Docker + Nginx* — reproducible deployment, TLS termination, horizontal scaling behind a load balancer.

14. Application Layers

1. **Frontend Presentation Layer** — screens, components, animations.
2. **Frontend State Layer** — session token, room state, socket event reducer.
3. **API Gateway / Controller Layer** — REST + WS entry points, auth/validation.
4. **Application/Service Layer** — orchestrates engines, enforces business rules.
5. **Domain Layer (Engines)** — Question, Challenge, Game, Room, Scoring, Report — see §15-25.
6. **Real-Time Communication Layer** — Socket.IO gateway + Redis adapter.
7. **Persistence Layer** — PostgreSQL via ORM.
8. **Cache Layer** — Redis (room state, rate limits, short-lived tokens).
9. **Security Layer** — token issuance/verification, PIN hashing, access control.
10. **Analytics/Logging Layer** — structured, PII-minimized event logs.
11. **Future AI Integration Layer** — pluggable provider behind Question/Challenge interfaces.

Data flows top-down for commands (Client → Controller → Service → Engine → DB) and bottom-up for events (Engine state change → Service → WS Gateway → Redis pub/sub → all instances → connected clients).

15. Functional Modules Overview

Room Management · Player/Identity Management · Game Mode Management · Consent & Preference Management · Game Engine · Question Engine · Challenge Engine · Roleplay Context Engine · Scoring Engine · Reporting Engine · Game History · Session Recovery · Real-Time Sync · (Future) Admin Portal.

16. Room Management

Room attributes: RoomID (UUID), SecureToken, CreatorSessionID, PartnerSessionID, OptionalPINHash, Status, CreatedAt, ExpiresAt, StartedAt, CompletedAt.

Rules:

- Token: cryptographically random, ≥ 128 bits entropy, URL-safe, single-use for the "second seat."
- Room capacity is exactly 2. A third visitor sees: *"This private room already has two players."*
- Default room expiry: 24 hours from creation if unstarted; configurable.
- Optional PIN (4–6 digits) is hashed (bcrypt/argon2) before storage; never stored/logged in plaintext.
- Once both seats are filled, the invitation URL no longer accepts new joiners.

17. Player / Identity Management (Lightweight, MVP)

- No registration required. Each player supplies a Nickname (1–20 chars, profanity-filtered) and confirms 18+.
- System issues a signed, short-lived **Session Token** (JWT, httpOnly secure cookie) binding: RoomID, PlayerSlot (A/B), DeviceFingerprintHash, IssuedAt, ExpiresAt.
- Architecture note: `TemporaryPlayers` table is deliberately separate from a future `Users` table so registered accounts can be introduced by adding a nullable `UserID` FK to `RoomPlayers` / `TemporaryPlayers` without schema rewrite.

18. Game Mode Management

- MVP modes: `LOVE_MAKING`, `ROLEPLAY` — modeled as rows in `GameModes`, not hard-coded enums, so future modes (Romantic, Deep Conversation, Funny Couple, Truth Challenge, Date Night, Relationship Memories, Fantasy, Couple Competition) are data, not code changes.
- **Sync flow:** Player A selects a mode $\rightarrow$ event `MODE_SELECTED` $\rightarrow$ Player B sees "Your partner selected X" with **Accept / Suggest Alternative**. Game starts only on `MODE_CONFIRMED` (both sides agree on the same ModeID).

19. Consent & Preference Management

- Mandatory 18+ attestation checkbox/screen for both players before room creation/join (stored as a timestamped boolean, not biometric/ID-verified).
- **Intensity Selection:** each player privately selects Soft / Playful / Intimate. $\text{EffectiveIntensity} = \text{MIN}(\text{PlayerA}, \text{PlayerB})$. Neither player sees the other's raw selection — only the resulting effective level — to avoid pressure signaling.
- Global consent controls available at all times in-game: Skip, Request Replacement, Pause, End Game. None of these affect score.

20. Game Engine

Owns the overall session lifecycle and orchestrates Question Engine, Challenge Engine, and Scoring Engine per round.

Exposes: `startGame`, `submitAnswer`, `selectChallenge`, `respondChallenge`, `pauseGame`, `resumeGame`, `completeGame`.

21. Question Engine

- Each player receives **10 questions**; question sets differ between players wherever the pool allows.
 - Difficulty curve: Q1–3 Easy, Q4–7 Medium, Q8–10 Medium/Hard.
 - Categories: Science, History, Geography, General Knowledge, Technology, Nature, Space, Entertainment, Problem Solving, Logical Reasoning, Basic Mathematics, Puzzles, Human Body, World Facts, Culture.
 - Content filter excludes: controversial, discriminatory, dangerous, highly political, religiously provocative, or over-specialized items.
 - MVP format: single-answer MCQ (4 options).
 - **QuestionProvider abstraction:**
-

```
interface QuestionProvider {
    getQuestions(category?, difficulty?, excludeIds?, count): Question[]
}
```

Implementations: `DatabaseQuestionProvider` (MVP), `AIQuestionProvider` (future), `AdminQuestionProvider` (future).

- Questions actually served in a session are **persisted at session start** (snapshotted into `QuestionAttempts`) so history/report stay reproducible even if the master bank changes later.
- Validation on ingestion: rejects duplicates, missing/ambiguous correct answer, malformed option sets, disallowed content categories.

22. Challenge Engine

Triggered on an incorrect answer. Generates **4 options** matching: selected GameMode, EffectiveIntensity (never exceeded), and variety rules (excludes recently shown ChallengeIDs for this session, avoids immediate repeats).

Challenge attributes: ChallengeID, GameMode, ChallengeType

(Describe/Answer/Perform/Choose/Improvise/Roleplay/Partner Prompt/Couple Conversation), Title, Instruction, IntensityLevel, Category, EstimatedDuration, RequiresResponse, RequiresPhysicalAction, RequiresDescription, SkipAllowed, ReplacementAllowed, ActiveStatus, CreatedBy, SourceType (`SYSTEM_GENERATED` | `ADMIN_CREATED`).

ChallengeProvider abstraction mirrors QuestionProvider: `SystemChallengeProvider` (MVP), `AdminChallengeProvider`, `AIChallengeProvider` (future), selectable by GameMode/Intensity/PreviousHistory/PlayerPreferences/SessionContext.

Resolution outcomes: `Completed`, `Skipped`, `Replaced` — all are valid, non-penalized terminal states for a challenge instance. Product copy always uses "Challenge"; internal traceability may retain the label "Incorrect Answer Challenge" for reporting only.

Content policy: No photo/video/audio capture or upload in MVP. For text-response challenges, only the typed response text is stored, tied to the challenge instance — no biometric or media data at any point.

23. Roleplay Context Engine (Roleplay Mode only)

Maintains continuity across roleplay challenges within one session:

```
RoleplayContext {
    sessionId, scenario, characterA, characterB,
    priorDecisions[], completedPrompts[], currentStoryState
}
```

When a new roleplay challenge is generated, the engine may reference the existing context to keep character/scenario continuity (e.g., Challenge 2 continues the scenario Challenge 1 established). Designed to accept a future AI narrative layer without schema change (context stored as JSONB).

24. Scoring Engine

- MVP: Correct = **+10**, Incorrect = **0**. Independent running totals for Player A and Player B.
- Challenge completion/skip/replacement **never** affects score — decoupling consent-activity outcomes from competitive pressure (BR-010).
- Config-driven for future extensions (difficulty multiplier, speed bonus, streak bonus) via a `ScoringConfig` table rather than hard-coded constants.

```
function scoreAnswer(isCorrect, config):
  if not isCorrect: return 0
  base = config.correctPoints # default 10
  bonus = 0
  if config.speedBonusEnabled: bonus += speedBonus(responseTimeMs, config)
  if config.difficultyBonusEnabled: bonus += difficultyBonus(difficulty, config)
  if config.streakBonusEnabled: bonus += streakBonus(currentStreak, config)
  return base + bonus
```

- End state: `Winner = A | B | Draw`. UI: "You Won ❤️" or "It's a Tie — Perfect Match ❤️".

25. Reporting Engine

Consumes the full session record and emits a structured JSON payload first, then renders UI from it (enabling future PDF export / shareable report / AI summary without a data-model change):

```
{
  gameSummary: {...}, players: {...}, scores: {...},
  knowledgeStats: {...}, categoryStats: {...},
  challengeStats: {...}, roleplaySummary: {...},
  winner: "...", timeline: [...]
}
```

Report includes: Game/Room ID, date/start/end/duration, both players, mode, effective intensity, scores, winner, totals, per-player correct/incorrect, accuracy %, category performance, average response time, challenges triggered/selected/completed/skipped/replaced, plus a playful, tasteful narrative ("Tonight's Game Story") and superlatives (Knowledge Champion, Fastest Thinker, Best Category, Most Difficult Category, Closest Round, Challenge Summary).

26. Game History

`GameSessions` record: SessionID, RoomID, Players, Mode, Status, CreatedAt/StartedAt/CompletedAt, all QuestionAttempts, Answers, Scores, Challenges + selections/responses, Skip/Replacement events, Winner, generated Report reference.

Status values: `ROOM_CREATED, WAITING_FOR_PARTNER, PARTNER_JOINED, WAITING_FOR_READY, MODE_SELECTION, BOUNDARY_SELECTION, IN_PROGRESS, PAUSED, COMPLETED, ABANDONED, EXPIRED`.

27. Detailed Functional Requirements (sample set — full matrix in §56)

ID	Requirement	Priority	Actor	Precondition	Trigger	System Behavior	Postcondition	Exception
FR-RM-001	Create private room	Must	Player A	18+ confirmed	Tap "Create Room"	Generate RoomID+SecureToken, persist, return link	Room in ROOM_CREATED	Token collision → regenerate
FR-RM-002	Join via link	Must	Player B	Room not full, not expired	Open link, enter nickname	Assign seat B, validate PIN if set	Room in PARTNER_JOINED	Wrong PIN → error, no lockout counter leak
FR-QE-001	Serve round question	Must	Game Engine	Game IN_PROGRESS	Round advance	Fetch/snapshot next question per player	QuestionAttempts row created	Provider failure → fallback pool
FR-CE-001	Generate challenge options	Must	Challenge Engine	Answer incorrect	ANSWER_SUBMITTED incorrect	Select 4 eligible challenges within intensity	Options returned to client	Pool exhausted → widen variety window
FR-SC-001	Compute score	Must	Scoring Engine	Answer submitted	Round resolved	Apply scoring config	Score updated	N/A
FR-REP-001	Generate report	Must	Report Engine	Game COMPLETED	Last round resolved	Aggregate session data to JSON	Report persisted & rendered	Partial data → mark report DEGRADED
FR-SEC-001	Isolate room access	Must	API Gateway	Any request	Any room-scoped call	Verify session token RoomID match	Request allowed/denied	Mismatch → 403 + audit log

(Full FR/NFR/BR sets follow the same template across all modules — Room, Player, Mode, Consent, Question, Challenge, Roleplay, Scoring, Report, History, Recovery, Real-Time, Security, Privacy, Accessibility.)

28. Business Rules

- BR-001 — A room contains at most two players.
- BR-002 — Both players must confirm 18+ before creating/joining.
- BR-003 — The game cannot start until both players are Ready.
- BR-004 — Game mode requires mutual confirmation of the same ModeID.
- BR-005 — Effective intensity = the lower of the two mutually selected levels.
- BR-006 — Each player receives exactly ten questions per game.
- BR-007 — An incorrect answer always triggers a challenge selection step.
- BR-008 — Exactly four eligible challenge options are generated per trigger.

- BR-009 — Either player may skip or reject any challenge at any time.
- BR-010 — Rejecting/skipping an activity never reduces score or is flagged negatively.
- BR-011 — No challenge offered may exceed the session's Effective Intensity.
- BR-012 — A third visitor to a full room is denied entry with a neutral message.
- BR-013 — PINs are never stored or logged in plaintext.
- BR-014 — Media capture (photo/video/audio) is not permitted anywhere in MVP.
- BR-015 — Challenge/typed response content is never written to application/debug logs.
- BR-016 — A session recovers in place on reconnect; it is never silently restarted.
- BR-017 — Reports are generated exactly once per completed game (idempotent generation).
- BR-018 — Only the two session-bound players may access a given room's data.
- BR-019 — Rooms expire automatically if unstarted within the configured window.
- BR-020 — Draws are a valid, positively-framed outcome, not a failure state.

29. User Stories (representative)

Room Creator

As a room creator, I want to generate a private secure link, so that only my partner can join our session. AC: Given I confirm 18+, when I tap Create Room, then a unique link is generated and the room enters `ROOM_CREATED`, and no other user can obtain seat A.

Joining Partner

As a joining partner, I want to enter the invitation link and join instantly, so that we can start playing without registration. AC: Given a valid, unexpired, non-full room link, when I confirm 18+ and (if set) enter the correct PIN, then I am placed in seat B and both players see "Connected."

Both Players

As both players, we want the game to only start once we both confirm readiness and the same mode, so that neither of us feels rushed or overridden. AC: Given both are in the room, when both tap Ready and both confirm the same GameMode, then the Intensity Selection screen activates.

Reconnecting Player

As a player who lost connection, I want to resume exactly where I left off, so that the session isn't disrupted. AC: Given a valid unexpired session token, when I reopen the app, then my current question/round/score/challenge state is restored and my partner is notified I've reconnected.

Future Admin

As an admin, I want to deactivate an inappropriate question, so that it stops appearing in new sessions immediately. AC: Given I am authenticated as Admin, when I deactivate a QuestionID, then it is excluded from all future `QuestionProvider` results without affecting past history.

30. Acceptance Criteria (platform-level sample)

```
Given Player A created a valid room
And Player B has not yet joined
When Player B accesses the valid invitation URL
Then Player B should be able to join
And both players should see Connected status
And no third participant should subsequently be allowed.
```

Given a round is active
When Player A submits an answer
Then Player A's answer is recorded and hidden from Player B
And the round only resolves once Player B has also answered.

31. Screen Specifications (representative subset — full set covers all 25 screens in §22 of the source brief)

Landing Page

- **Purpose:** entry point; explains the product briefly, premium tone.
- **Fields:** none required.
- **Buttons:** Create a Room, I Have a Link (Join).
- **Validation:** n/a. **Empty:** default marketing copy. **Loading:** skeleton. **Error:** offline banner. **Mobile:** single-column, thumb-reachable CTAs.

Age Confirmation

- **Fields:** attestation checkbox + statement text.
- **Buttons:** Confirm & Continue (disabled until checked), Exit.
- **Validation:** checkbox required. **Error:** inline "Please confirm to continue."

Create Room / Join Room

- **Fields:** Nickname (required, 1-20 chars), Optional PIN (Create only, 4-6 digits), PIN entry (Join only, if room has one).
- **Buttons:** Create Room / Join Room, Back.
- **Validation:** nickname profanity filter; PIN numeric only.
- **Error states:** room full, room expired, wrong PIN (generic message, no attempt counter shown to prevent enumeration hints).

Waiting Room / Partner Connected / Ready

- **Displayed:** Player A/B connection dots, "Your partner has joined," Ready toggle for self and partner status.
- **Loading:** pulsing "waiting for partner" indicator. **Mobile:** large single Ready button.

Game Mode Selection

- Mode cards (Love Making, Roleplay) with short neutral descriptions; partner's pending selection shown as "Your partner selected X — Accept / Suggest Alternative."

Boundary / Intensity Selection

- Three level cards (Soft/Playful/Intimate), private selection, no visibility into partner's raw pick, reveal only the resulting Effective Level, with calm, non-judgmental copy.

Question Screen / Answer Feedback / Wrong Answer

- Question text, 4 options (large tap targets), submit locks the option; feedback screen shows Correct/Incorrect with score delta; wrong answer transitions directly into Challenge Options.

Challenge Options / Details / Response

- 4 challenge cards (title + short teaser); detail view on tap with full instruction, Skip/Replace/Accept controls; response view depends on `RequiresResponse` (text field, or Mark Complete button).

Round Summary / Progress

- "Round 4/10," both scores, progress bar; never reveals partner's answer pre-submission — shows "You answered ✓ / Waiting for partner..." until both are in.

Pause / Disconnected Partner

- Pause: both must resume to continue (or either can end). Disconnected: "Your partner's connection dropped — waiting to reconnect," with a visible timeout and graceful exit path.

Final Result / Winner / Game Report / Game History / Settings / Privacy Controls

- Scores, winner banner, full report (per §25), history list of past sessions (metadata only, no explicit content preview), and privacy controls to delete a session's history/report/responses.

(Every screen consistently defines Purpose, Fields, Buttons, Actions, Validation, Empty/Loading/Error states, and Mobile behavior per the template above; the remaining screens follow identical structure.)

32. UI/UX Design System

Direction: Romantic, modern, private, premium, warm, minimal — never explicit or juvenile.

Palette (dark-mode primary):

Token	Hex	Use
Background	#12090D	App base
Surface	#211016	Cards/panels
Primary Accent	#9D2945	Primary actions
Secondary	#E58A9A	Secondary highlights
Gold Accent	#D6B26E	Premium accents, badges
Text	#FFF7F4	Primary text

Effects: soft gradients, light glassmorphism, subtle glow, smooth card transitions, occasional heart particles, elegant micro-interactions — all GPU-cheap (transform/opacity only), never blocking interaction.

Accessibility overlay: never encode meaning by color alone (correct/incorrect/selected/disconnected all pair an icon/label with color); reduced-motion variant for all animated transitions; WCAG AA contrast minimum.

33. Mobile Responsiveness

Breakpoints: Mobile 320–767px, Tablet 768–1023px, Desktop 1024px+. Primary target viewports: 360×800, 390×844, 412×915. No horizontal scroll. Minimum touch target ≈44px. One-hand reachable primary actions (bottom-anchored CTAs).

34. Database Design

Core entities (MVP): Users (future), TemporaryPlayers, Rooms, RoomPlayers, Games, GamePlayers, GameRounds, Questions, QuestionOptions, QuestionAttempts, GameAnswers, GameModes, Challenges, ChallengeOptions, ChallengeSelections, ChallengeResponses, RoleplayContexts, GameScores, GameReports, PlayerSessions, ConsentPreferences, GameEvents, AuditLogs.

Future: Admins, Roles, Permissions, AdminQuestions, ContentModerationLogs.

Representative table definitions

Rooms

Column	Type	Constraints
id	UUID	PK
secure_token	VARCHAR(64)	UNIQUE, NOT NULL, indexed
creator_session_id	UUID	FK → PlayerSessions, NOT NULL
partner_session_id	UUID	FK → PlayerSessions, NULLABLE
pin_hash	VARCHAR(255)	NULLABLE
status	ENUM(RoomStatus)	NOT NULL, default ROOM_CREATED
created_at	TIMESTAMP TZ	NOT NULL, default now()
expires_at	TIMESTAMP TZ	NOT NULL
started_at	TIMESTAMP TZ	NULLABLE
completed_at	TIMESTAMP TZ	NULLABLE

GameSessions (Games)

Column	Type	Constraints
id	UUID	PK
room_id	UUID	FK → Rooms, NOT NULL
mode_id	UUID	FK → GameModes, NOT NULL
effective_intensity	ENUM(Soft,Playful,Intimate)	NOT NULL
status	ENUM(GameStatus)	NOT NULL
created_at / started_at / completed_at	TIMESTAMP TZ	as applicable

QuestionAttempts

Column	Type	Constraints
id	UUID	PK
game_id	UUID	FK → Games
player_slot	ENUM(A,B)	NOT NULL
question_id	UUID	FK → Questions (snapshot reference)
sequence_number	SMALLINT	1-10, NOT NULL
selected_option_id	UUID	NULLABLE until answered
is_correct	BOOLEAN	NULLABLE until answered
response_time_ms	INTEGER	NULLABLE
answered_at	TIMESTAMP TZ	NULLABLE
UNIQUE(game_id, player_slot, sequence_number)		prevents duplicate rounds

Challenges

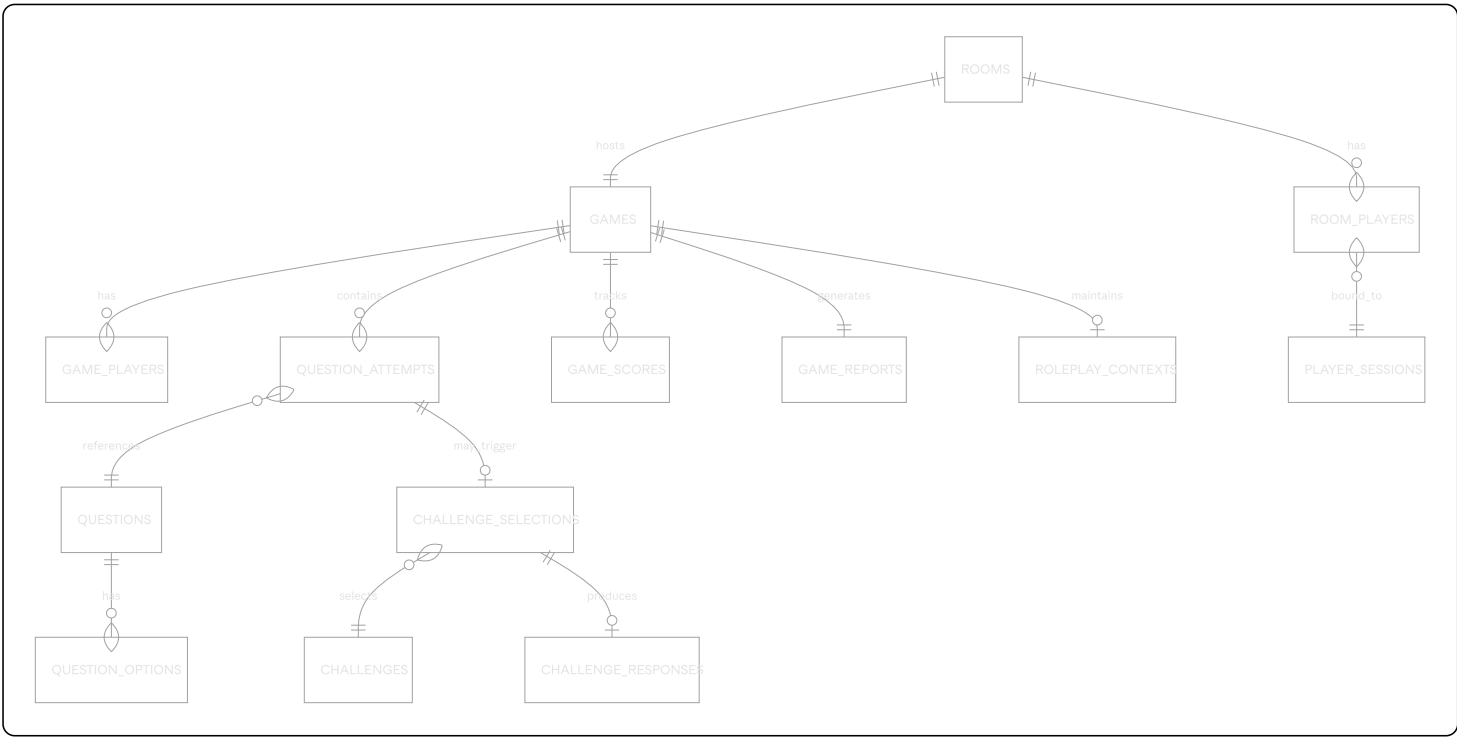
Column	Type	Constraints
id	UUID	PK
game_mode_id	UUID	FK → GameModes
challenge_type	ENUM	NOT NULL
title / instruction	TEXT	NOT NULL
intensity_level	ENUM(Soft,Playful,Intimate)	NOT NULL, indexed
category	VARCHAR	NULLABLE
estimated_duration_sec	INTEGER	NULLABLE
requires_response / requires_physical_action / requires_description	BOOLEAN	NOT NULL default false
skip_allowed / replacement_allowed	BOOLEAN	NOT NULL default true
active_status	BOOLEAN	NOT NULL default true
source_type	ENUM(SYSTEM_GENERATED, ADMIN_CREATED)	NOT NULL

ChallengeResponses

Column	Type	Constraints
id	UUID	PK
challenge_selection_id	UUID	FK → ChallengeSelections
response_text	TEXT	NULLABLE (only if <code>requires_response</code>)
outcome	ENUM(Completed,Skipped,Replaced)	NOT NULL
created_at	TIMESTAMP TZ	NOT NULL

All tables include `created_at`/`updated_at`; identity-bearing/sensitive tables (`TemporaryPlayers`, `ChallengeResponses`, `GameReports`) support soft delete (`deleted_at`) to satisfy user-initiated deletion (§37) while preserving referential integrity of aggregate, de-identified analytics.

ERD (core relationships)



35. API Specifications (representative — full CRUD/event matrix follows same pattern)

POST /rooms

- Auth: none (pre-session) | Request: `{nickname, ageConfirmed, pinOptional}` | Response: `{roomId, secureToken, inviteUrl, sessionToken}` | 201 | Errors: 400 invalid nickname | BR: BR-001, BR-002.

POST /rooms/{token}/join

- Auth: none | Request: `{nickname, ageConfirmed, pin?}` | Response: `{roomId, sessionToken, partnerNickname}` | 200 | Errors: 404 not found/expired, 409 room full, 401 wrong PIN | BR: BR-001, BR-012, BR-013.

GET /rooms/{token} — public-safe room status (no sensitive fields) for pre-join link preview.

POST /rooms/{id}/ready — Auth: session token bound to room | marks player ready; emits `PLAYER_READY`.

POST /games/{id}/mode — proposes/accepts a mode; emits `MODE_SELECTED` / `MODE_CONFIRMED`.

POST /games/{id}/preferences — submits intensity level privately; server computes/reveals only `effectiveIntensity` once both submitted.

POST /games/{id}/start — Auth: both players ready + mode confirmed + preferences set | transitions to `IN_PROGRESS`.

GET /games/{id}/question — returns current player's next question (never includes correct answer or partner's question).

POST /games/{id}/answers — `{questionAttemptId, selectedOptionId, responseTimeMs}` | idempotent (duplicate submission rejected, BR edge case §38) | triggers Challenge Engine on incorrect.

GET /games/{id}/challenge-options — 4 eligible challenges for the current trigger.

POST /games/{id}/challenge-selection / /challenge-response / /challenge-skip — record outcome; never mutate score.

POST /games/{id}/pause / /resume — Auth: either player | requires other player's acknowledgment to resume (configurable).

GET /games/{id}/result — final scores/winner once `COMPLETED`.

GET /games/{id}/report — full structured report (idempotent generation, BR-017).

All endpoints: request/response schemas versioned; standard error envelope `{code, message, correlationId}`; all room/game-scoped endpoints re-verify session-token-to-room binding server-side regardless of client-supplied IDs (BR-018).

36. WebSocket / Event Specifications

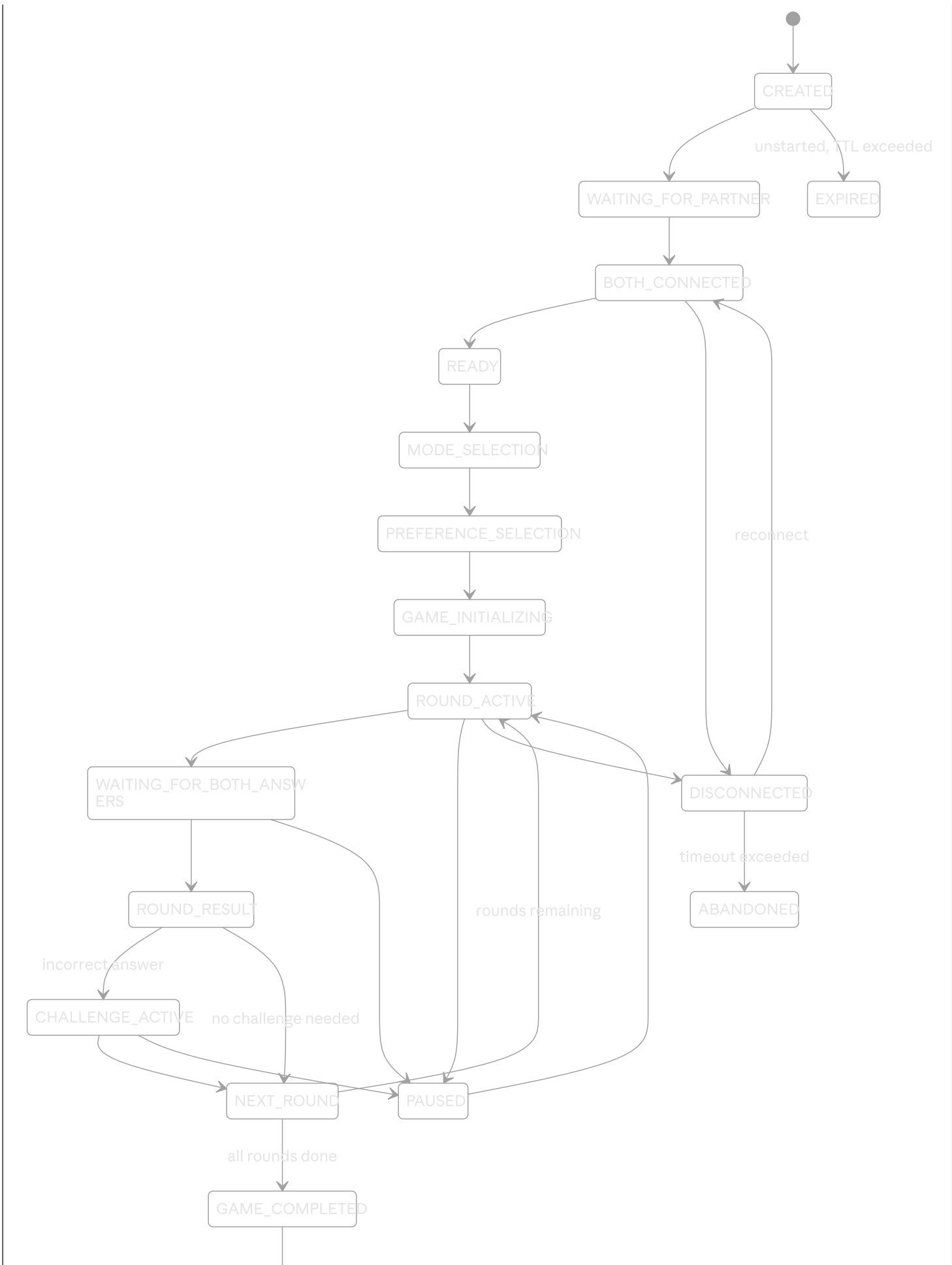
Transport: Socket.IO with Redis adapter for multi-instance pub/sub.

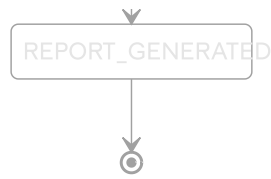
Events: `ROOM_CREATED, PLAYER_JOINED, PLAYER_READY, MODE_SELECTED, MODE_CONFIRMED, GAME_STARTED, QUESTION_ANSWERED, ROUND_COMPLETED, CHALLENGE_GENERATED, CHALLENGE_SELECTED, CHALLENGE_COMPLETED, CHALLENGE_SKIPPED, PLAYER_DISCONNECTED, PLAYER_RECONNECTED, GAME_PAUSED, GAME_RESUMED, GAME_COMPLETED`.

Payload principle: events carry only what the *recipient* is entitled to see — e.g., `QUESTION_ANSWERED` broadcast to the partner contains a boolean "partner has answered," never the answer itself, until `ROUND_COMPLETED` reveals both outcomes simultaneously.

Authorization: every socket connection authenticates via the same session token used for REST; the gateway joins the socket to a room channel named by RoomID only after verifying binding, preventing cross-room event leakage.

37. State Machine





38. Sequence Diagrams (representative)

Incorrect Answer → Challenge

```

sequenceDiagram
    participant A as Player A Client
    participant S as Backend
    participant B as Player B Client

    A->>S: POST /games/{id}/answers
    S->>S: Evaluate answer (incorrect)
    S->>S: Challenge Engine selects 4 options (mode+intensity+history)
    S-->>A: Round result + Challenge options
    S-->>B: ROUND_COMPLETED (partner incorrect, no content leak)
    A->>S: POST /games/{id}/challenge-selection
    S-->>B: CHALLENGE_SELECTED (context-appropriate visibility)
    A->>S: POST /games/{id}/challenge-response
    S->>S: Persist outcome (Completed/Skipped/Replaced), score unaffected
    S-->>A: NEXT_ROUND
    S-->>B: NEXT_ROUND
  
```

Reconnect

```

sequenceDiagram
    participant P as Player Client
    participant S as Backend
    P->>S: Reopen app (session token cookie)
    S->>S: Validate token, load room/game/round state from PG+Redis
    S-->>P: Current state snapshot (question, score, round, challenge state)
    S-->>P: partner: PLAYER_RECONNECTED
  
```

(Create Room, Join Room, Start Game, Challenge Selection detail, Next Round, Complete Game, Generate Report follow the same clarity level.)

39. Security Requirements

NFR-SEC-001 HTTPS only, HSTS enabled. NFR-SEC-002 Secure, httpOnly, SameSite=Strict cookies for session tokens. NFR-SEC-003 CSRF protection on all state-changing REST endpoints. NFR-SEC-004 Output encoding / CSP to prevent XSS. NFR-SEC-005 Parameterized queries/ORM only — no raw SQL concatenation. NFR-SEC-006 Rate limiting on room create/join/PIN attempts (per-IP and per-token). NFR-SEC-007 All room/game identifiers are non-sequential UUIDs. NFR-SEC-008 PINs hashed with a modern KDF (argon2id/bcrypt), never logged. NFR-SEC-009 Session tokens short-lived with sliding refresh, bound to RoomID+PlayerSlot. NFR-SEC-010 TLS in transit; encryption at rest for the database volume. NFR-SEC-011 Secrets in a managed secrets store, never in source/config. NFR-SEC-012 Structured audit log for room access, PIN failures, cross-room access attempts. NFR-SEC-013 Every REST/WS handler re-checks RoomID/PlayerSlot ownership server-side (defense against broken access control) — players can never reach another couple's room, report, or response data.

40. Privacy Requirements

Privacy-by-design: no CNIC/government ID/address/phone required in MVP. No camera/microphone access, no media storage, no biometric data at any layer. Room TTL and game-history retention are configurable parameters (not hard-coded). Players can delete a Game, its Report, and its stored Responses; deletion is enforced at the data layer (soft delete + scheduled hard purge) and reflected immediately in Game History. Challenge/response text is explicitly excluded from all application, security, and debug logs (BR-015) — logs reference only IDs and correlation IDs.

41. Performance Requirements

NFR-PERF-001 Landing page interactive <2s on typical 4G. NFR-PERF-002 Standard API p95 <500ms. NFR-PERF-003 Room-event propagation <300ms where infrastructure permits. NFR-PERF-004 Screen-to-screen game transitions feel near-instant (<150ms perceived). NFR-PERF-005 Architecture supports horizontal scaling via stateless API nodes + Redis-backed WS adapter for a defined initial concurrent-session target, scaling out before scaling up.

42. Zero-Lag Mobile Experience

Animations restricted to `transform`/`opacity`; avoid heavy DOM churn, large JS bundles, uncompressed media, and excessive particle effects. Techniques: code splitting per route, lazy loading of non-critical assets, pre-fetch of the next question and next challenge metadata during idle time, optimized/subset fonts, compressed images, HTTP caching headers.

43. Accessibility

Readable font sizes and line-height, WCAG AA contrast, ≥44px touch targets, keyboard operability where applicable on desktop, screen-reader labeling for all interactive controls, `prefers-reduced-motion` support, and state indication (correct/incorrect/selected/disconnected) always paired with icon/text, never color alone.

44. Error Handling

Standard error envelope across REST (`{code, message, correlationId}`) and WS (`ERROR`) event with the same shape). User-facing copy stays neutral and non-blaming (e.g., PIN failure never reveals whether the room exists). All errors are correlation-ID tagged for support/debugging without exposing sensitive payloads.

45. Edge Cases

Scenario	Expected Behavior
Partner never joins	Room sits <code>WAITING_FOR_PARTNER</code> until TTL, then <code>EXPIRED</code>
Room expires	Join attempts return 404/expired state, neutral copy
Third person opens link	Denied with "room already has two players"
Wrong PIN	Generic rejection, rate-limited, no enumeration signal
Link reused after room full	Rejected, same as third-person case
One player refreshes	Session token restores state in place (§ Session Recovery)
Both refresh	Both restore independently; sync re-established via WS rejoin
Internet disconnect	<code>PLAYER_DISCONNECTED</code> broadcast; game <code>PAUSED</code> or held, grace window before <code>ABANDONED</code>
Player switches device	New device presenting a valid session token resumes normally
Player leaves mid-game	Partner sees Disconnected state with exit option; no auto-forfeit scoring
Player stays offline past grace window	Session moves to <code>ABANDONED</code> ; partial report optionally generated
Server restarts	Redis/PG-backed state allows resumption once instance is back
Socket disconnects (transient)	Socket.IO auto-reconnect; state re-synced from server truth, not client cache
Question generation fails	Provider fallback pool; if fully exhausted, graceful "please retry" with no game-breaking crash
Challenge generation fails	Same fallback pattern within Challenge Engine
Duplicate answer submission	Second submission for same <code>(game, slot, sequence)</code> rejected idempotently
Double-tap → two answers	Client-side debounce + server-side unique constraint prevents double write
Submit after round already completed	Rejected with "round already resolved"
Skip challenge	Recorded as <code>Skipped</code> , no score impact, proceeds to next round
Multiple replacement requests	Allowed within configured cap; variety engine avoids repeats
Game abandoned	Status <code>ABANDONED</code> ; no winner computed; history retained per retention policy
Both players receive same question accidentally	Prevented by per-slot exclusion list at generation time; treated as a defect if it occurs
Game ends in draw	Valid outcome, positively framed
Session token stolen	Token is short-lived, device-fingerprint-bound, and revocable; audit log flags anomalous slot access
Expired room accessed	Neutral "no longer available" response

Scenario	Expected Behavior
Report generated twice	Generation is idempotent — a second call returns the existing report, never duplicates it

46. Logging & Monitoring

Layers: Application Logs, Security Logs, Error Logs, Game Event Logs, Audit Logs — each carrying a correlation ID that threads a request across services. Sensitive content (challenge responses, PINs) is excluded by design from every log sink, enforced at the logging-middleware level, not left to developer discipline alone.

47. Analytics

Operational, non-sensitive metrics only: games created/started/completed, completion rate, most-played mode, average duration, question accuracy, hardest categories, challenge skip rate. Private challenge-response content is never surfaced in analytics dashboards or exports.

48. Testing Strategy

Unit, Integration, API, WebSocket, Database, Security, Performance, Mobile-responsive, Cross-browser, End-to-End, and UAT layers.

High-priority scenarios: two users successfully join; third user rejected; both must select the same mode to proceed; questions remain hidden from partner pre-submission; an answer is processed exactly once; an incorrect answer always yields valid, intensity-compliant challenge options; a rejected challenge can be replaced without penalty; a refresh restores exact state; the final winner is computed correctly; the generated report matches the full game history event-for-event.

49. Deployment Architecture

Containerized services (Docker) behind Nginx reverse proxy and a load balancer; stateless API nodes scale horizontally; Redis and PostgreSQL as managed/clustered services; blue-green or rolling deploys to avoid mid-game disruption; health checks gate traffic cutover; suitable on AWS/Azure/GCP or a comparable container platform.

50. Folder Structure

```
/apps
  /web      → Next.js client: screens, components, socket client, state
  /api      → NestJS backend: controllers, gateways, services
/packages
  /shared   → shared TS types/DTOs/event contracts between web and api
  /game-engine → room/game orchestration, state machine
  /question-engine → QuestionProvider implementations + validation
  /challenge-engine → ChallengeProvider implementations + variety/intensity logic
/infrastructure → Dockerfiles, Nginx config, IaC, CI/CD pipelines
/docs      → this FSD and derived specs (API, ERD, diagrams)
```

Each engine package is independently testable and exposes only its interface (`QuestionProvider`, `ChallengeProvider`, etc.) to the Game Engine — the mechanism that lets future AI or admin-driven providers be swapped in without touching orchestration code.

51. MVP Scope (Recap)

Private room + invite link, two players, temporary identity, 18+ confirmation, mutual ready, two modes, mutual intensity, 10 questions/player, system questions, wrong-answer challenge engine, scoring, winner, report, session history, real-time sync, responsive mobile web.

52. Future Admin Portal (Architecture Only — Not Built in MVP)

Secure admin login; CRUD for Questions (category/difficulty/correct answer/options) and Challenges (mode/intensity/type); optional per-question "eligible challenge pool" mapping (e.g., Question X wrong → eligible ChallengeIDs list) as an alternative to the default mode/category/intensity-driven selection; management of Game Modes, Categories, Difficulty, Intensity; content activate/deactivate; aggregated, de-identified game analytics. Admins are architecturally prevented from casually accessing private couple responses — response content requires a distinct, heavily audited, justification-gated access path (or is excluded from admin visibility entirely in v1 of the portal).

53. Future Roadmap

- **Phase 2:** registered accounts, personal game-history dashboard, additional modes, question personalization, richer reports, AI question/challenge generation.
- **Phase 3:** full Admin Portal UI, content mapping tools, configuration UI, analytics dashboard.
- **Phase 4:** contextual AI-driven roleplay continuity, personalized difficulty, stored couple preferences, PWA/native app.

54. Risks & Mitigations

Risk	Mitigation
Sensitive content leakage via logs/analytics	Logging middleware hard-excludes response fields; analytics pipeline reads only aggregate tables
Link/token guessing or reuse	High-entropy tokens, single-seat consumption, TTL expiry, rate limiting
Pressure dynamics around intensity selection	Private selection, reveal only the resulting effective level, neutral UI copy
Session hijack via stolen token	Short TTL, device-fingerprint binding, httpOnly/secure cookies, anomaly auditing
Scope creep into explicit/media content	Explicit MVP prohibition on media capture/storage; content review gate for challenge library
Report generated inconsistently on retries	Idempotent report generation keyed by GameID
Multi-instance WebSocket state drift	Redis-backed Socket.IO adapter as single source of real-time truth

55. Open Configuration Parameters

Room TTL (default 24h), PIN length (4-6), history retention window, scoring config (base points, optional bonuses), challenge replacement cap per round, disconnect grace period before ABANDONED, content filter word-lists, per-environment rate-limit thresholds.

56. Requirement Traceability Matrix (structure)

Requirement ID	Business Rule(s)	Screen(s)	API/Event(s)	Test Case(s)
FR-RM-001	BR-001, BR-002	Create Room	POST /rooms	TC-RM-001
FR-RM-002	BR-001, BR-012, BR-013	Join Room	POST /rooms/{token}/join	TC-RM-002..004
FR-QE-001	BR-006	Question Screen	GET /games/{id}/question	TC-QE-001..003
FR-CE-001	BR-007, BR-008, BR-011	Challenge Options	GET/POST challenge-*	TC-CE-001..006
FR-SC-001	BR-010, BR-014	Round Summary	POST answers	TC-SC-001..002
FR-REP-001	BR-017	Game Report	GET /games/{id}/report	TC-REP-001
FR-SEC-001	BR-018	All screens	All endpoints	TC-SEC-001..010

(Every FR/BR pair in §27–28 maps into this matrix in the working traceability spreadsheet maintained alongside this document.)

57. Definition of Done (MVP)

- All §27 functional requirements implemented and passing §48 test suite (unit/integration/E2E/security).
- All §45 edge cases verified with automated regression tests.
- Security review sign-off against §39 checklist; privacy review sign-off against §40.
- Performance targets in §41 met under defined concurrent-session load test.
- Accessibility checklist (§43) passed on primary mobile viewports.
- Report output validated against real session data for at least 20 full game playthroughs (QA).
- Deployment pipeline (§49) green on staging with rollback verified.
- This FSD, ERD, API spec, and event catalog are versioned and stored in `/docs` and kept in sync with the shipped build.

End of Document.